

Demystifying the Data Lakehouse in Fabric

Just Blindbæk

Field CTO, Tabular Editor





THANK YOU TO OUR AMAZING SPONSORS !

glueck  kanja

robopack 
empowered by SOFTWARE
CENTRAL


nerdio



Continue

veeam

 VARONIS

eido

 Admin By Request
ZERO TRUST PLATFORM

Lenovo

 dynatrace

 VirtualMetric


Red Hat

 Arkimentum


semperis

DanofficeIT

 EPIC FUSION
BRING IT ALL TOGETHER

Recast

twoday

 European
Power Platform
Conference
CO-ROUNDED 19 JUNE - 22 JULY 2024

inforcer 


APENTO

TRUESEC

 GLOBE SYSTEMS

control  UP

 PATCH
MY PC

JN DATA
WE ARE ON

 systemcenterdudes

 EasyEntra

 Microsoft

 Implora.io

Yealink

 TD SYNnex

 TOKEN
token2.swiss

 Cloud Factory


GLOBETEAM

 NORDSTERN

VENZO_
a salantis company

FEITIAN
WE BUILD SECURITY

 identity stack

 2LINKIT



What is this session all about?

- What is this Data Lakehouse that everyone is talking about now?
- Why Bronze/Silver/Gold now?
- Why decoupling storage and compute now?
- Do we all need to learn Python?
- What about good old SQL?



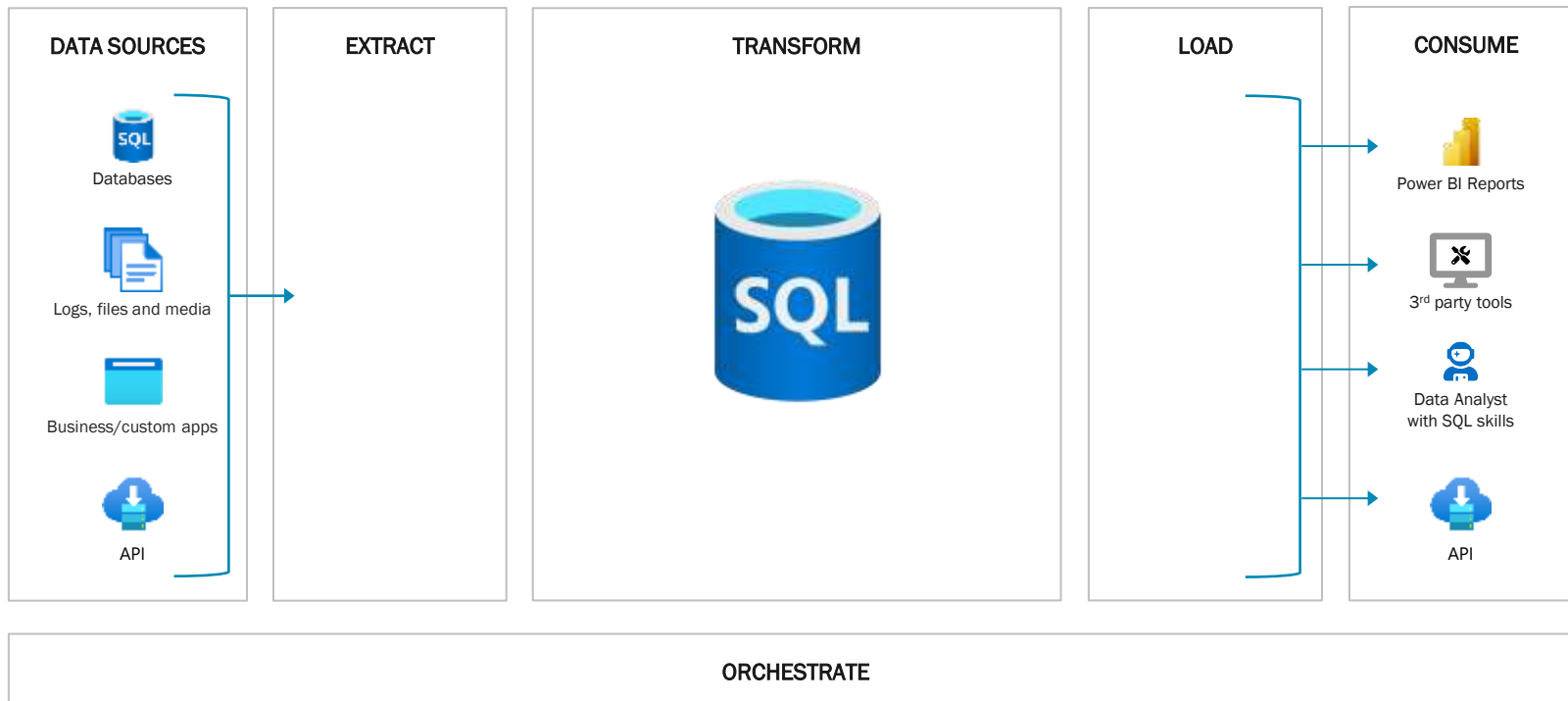


From Classic to Modern

Exploring Layered Data Platform Architectures

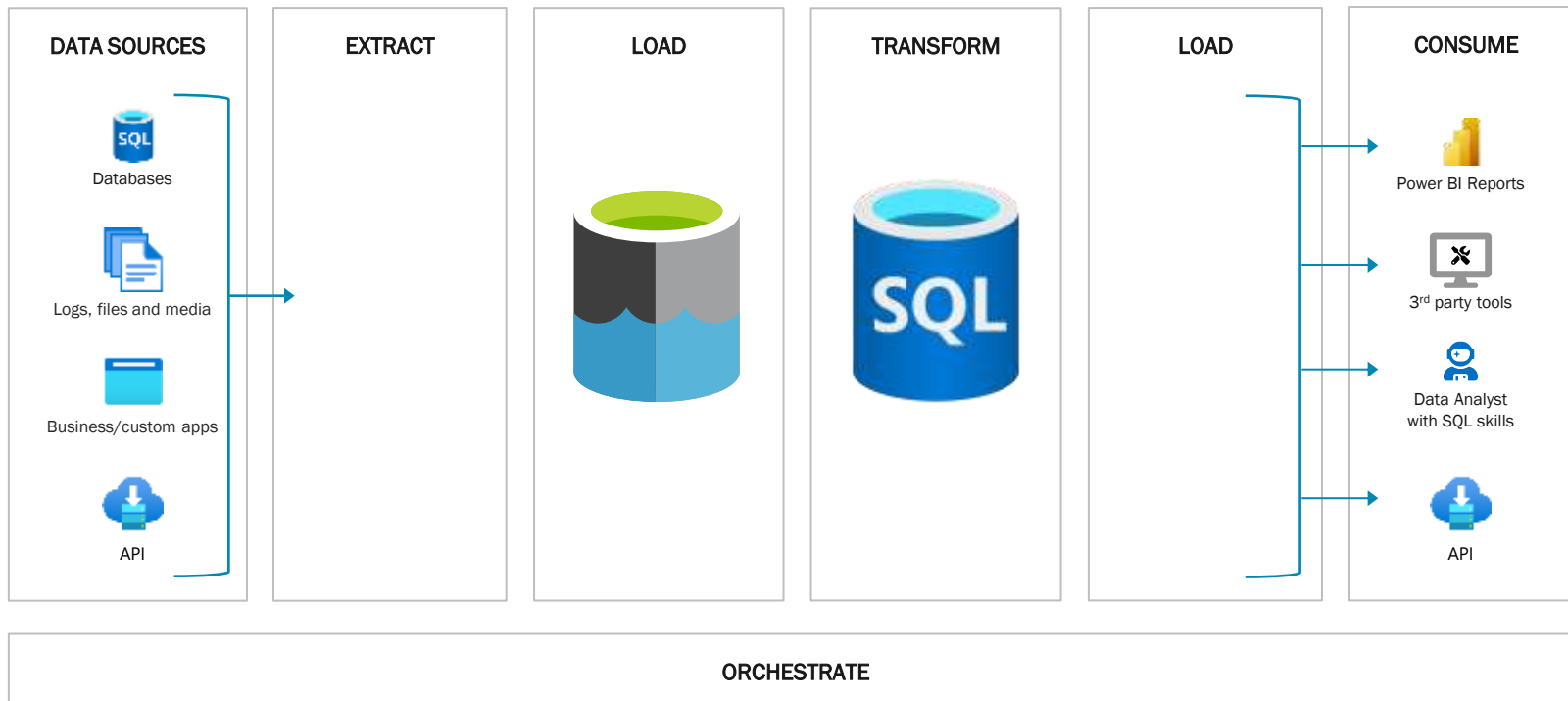


Dataflow in a Classic Data Warehouse



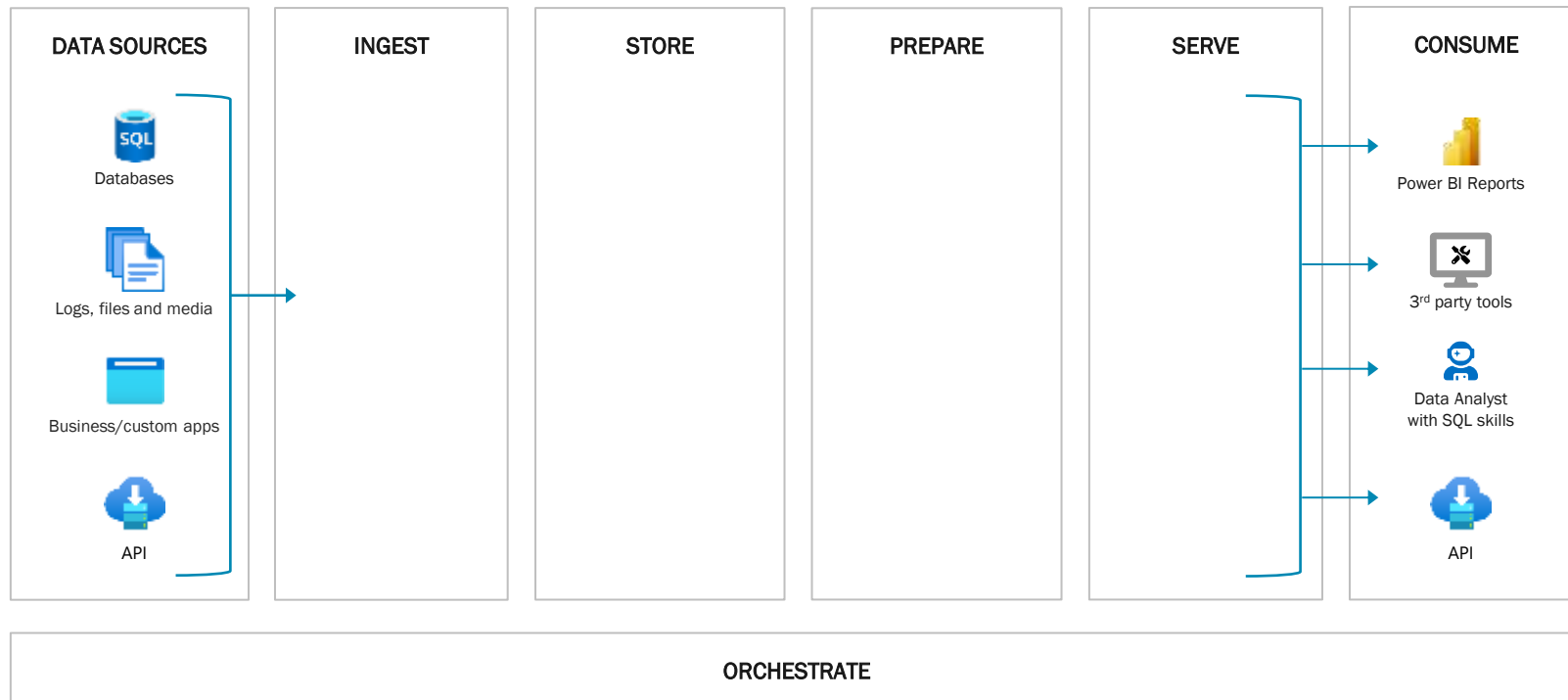


Dataflow in a Modern Data Warehouse



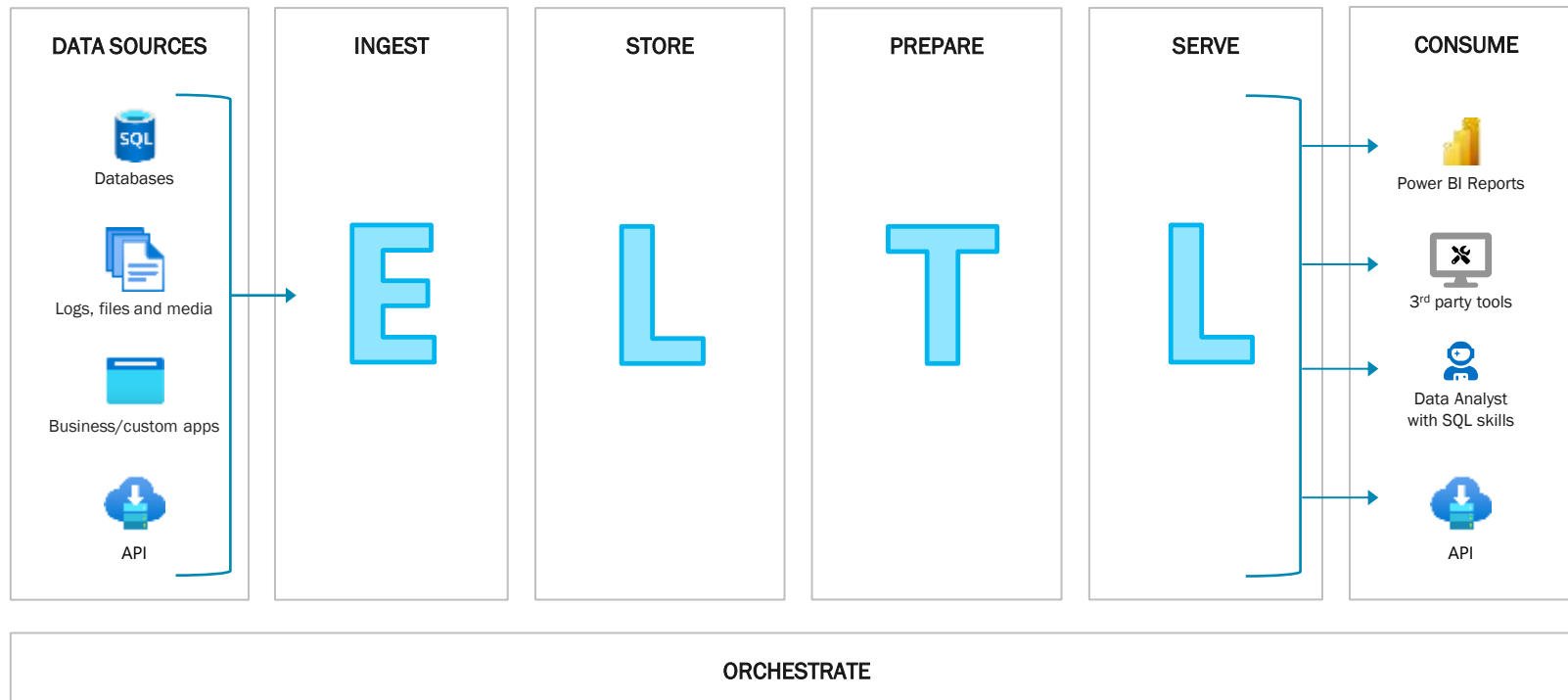


Dataflow in a Modern Data Warehouse





Dataflow in a Modern Data Warehouse

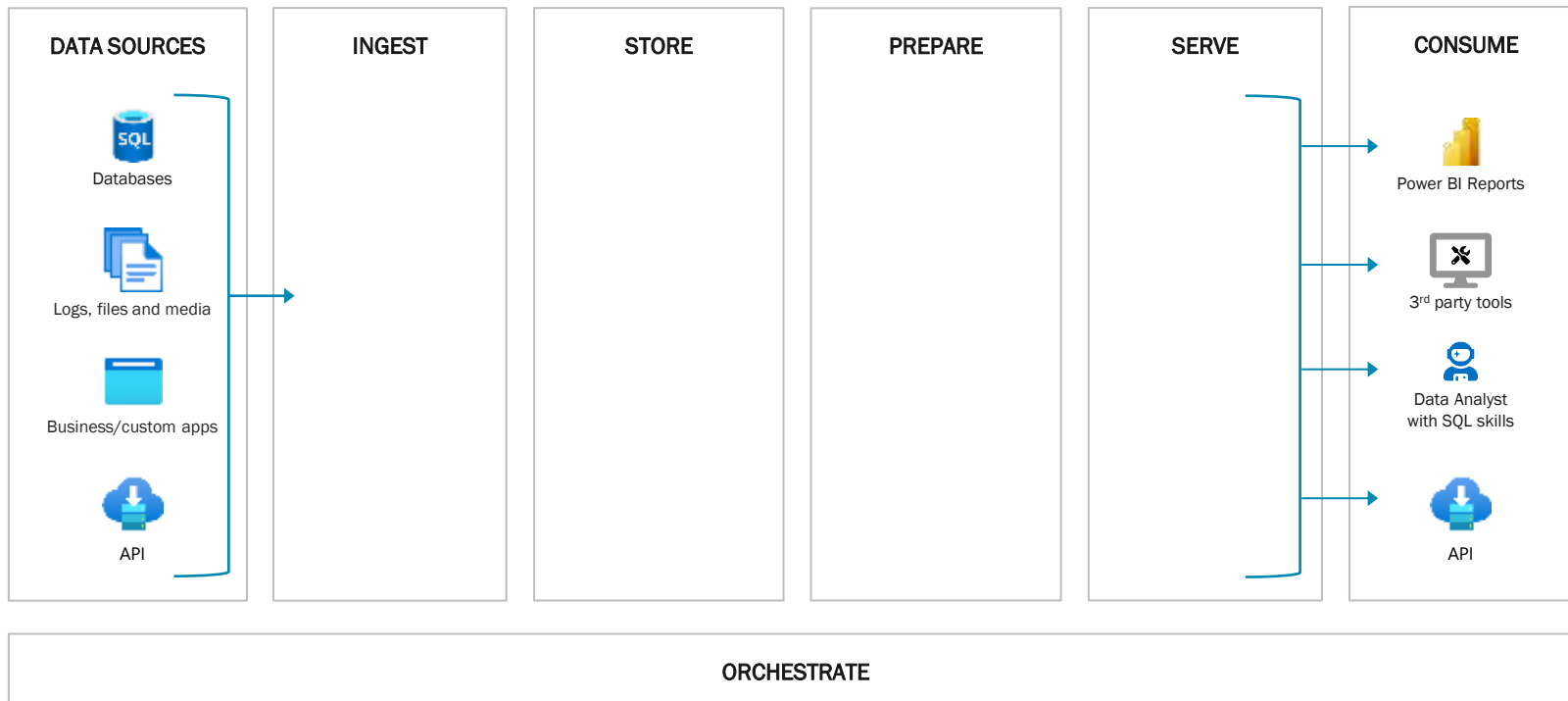




Unleashing the Power of Decoupled Storage and Compute

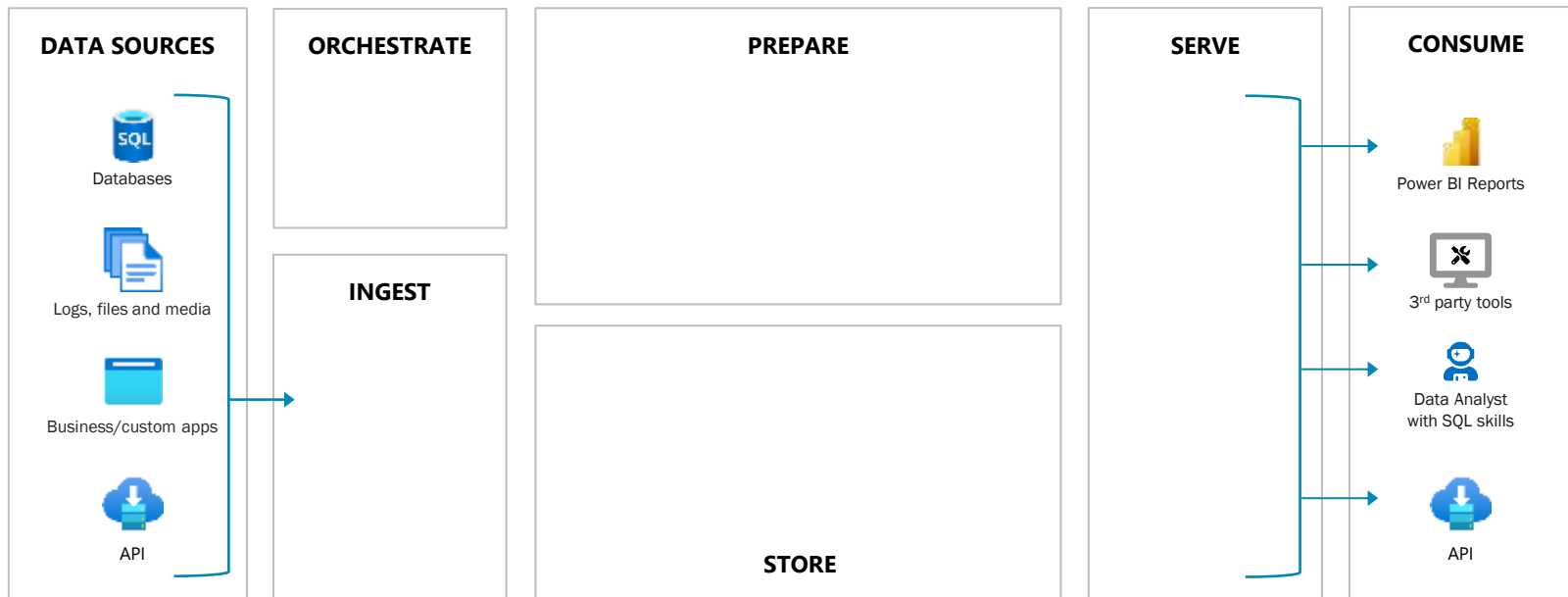


Dataflow in a Modern Data Warehouse



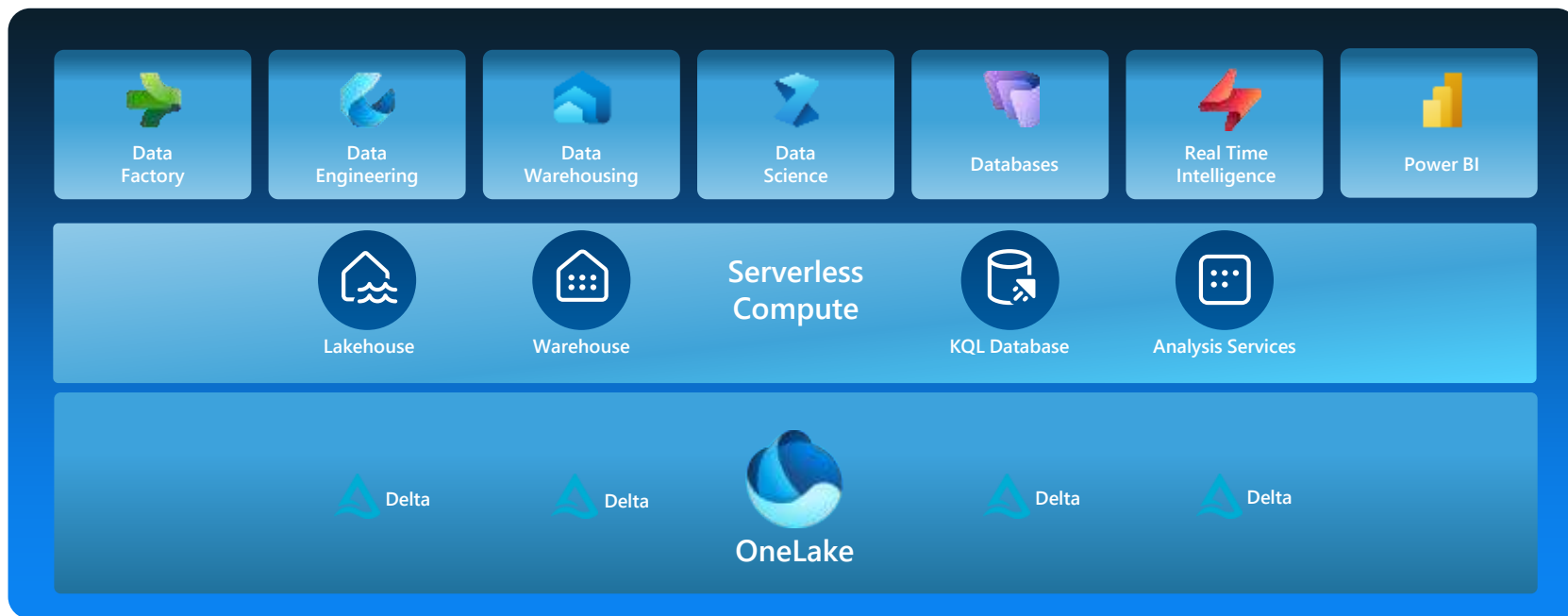


Dataflow in a Data Lakehouse architecture



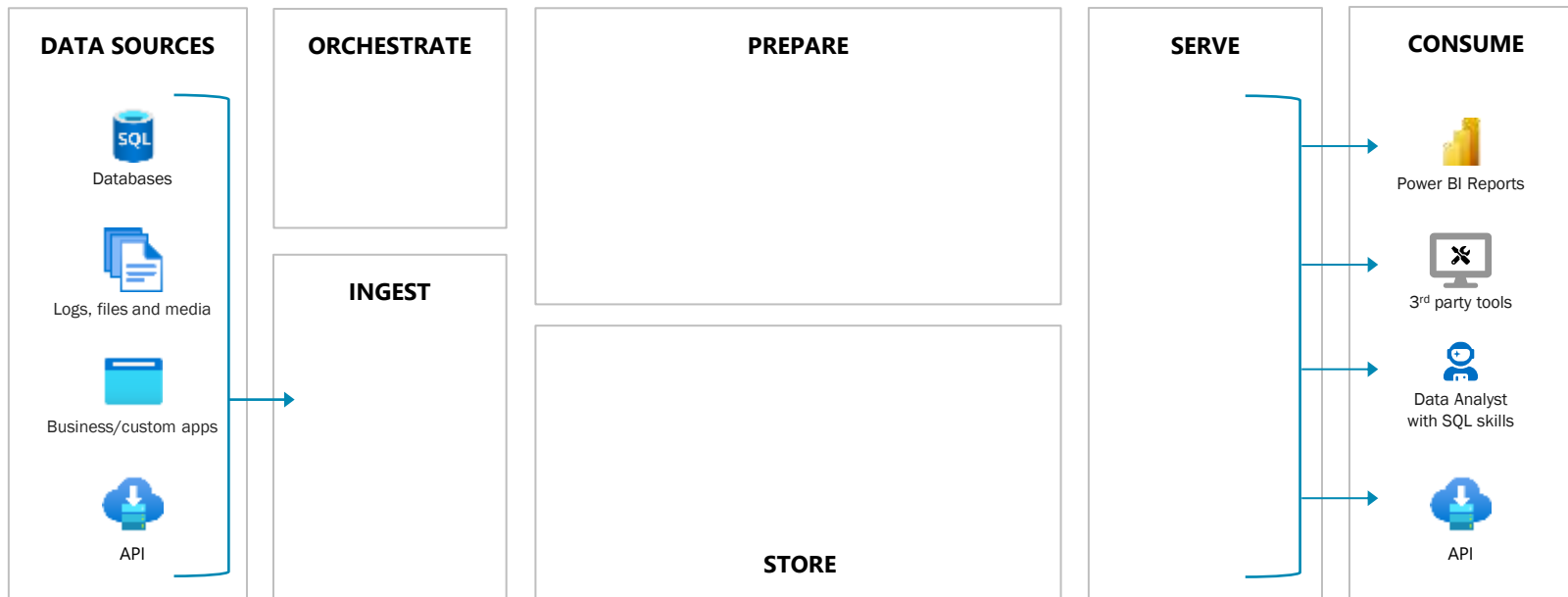


Fabric as an end-to-end unified analytical platform



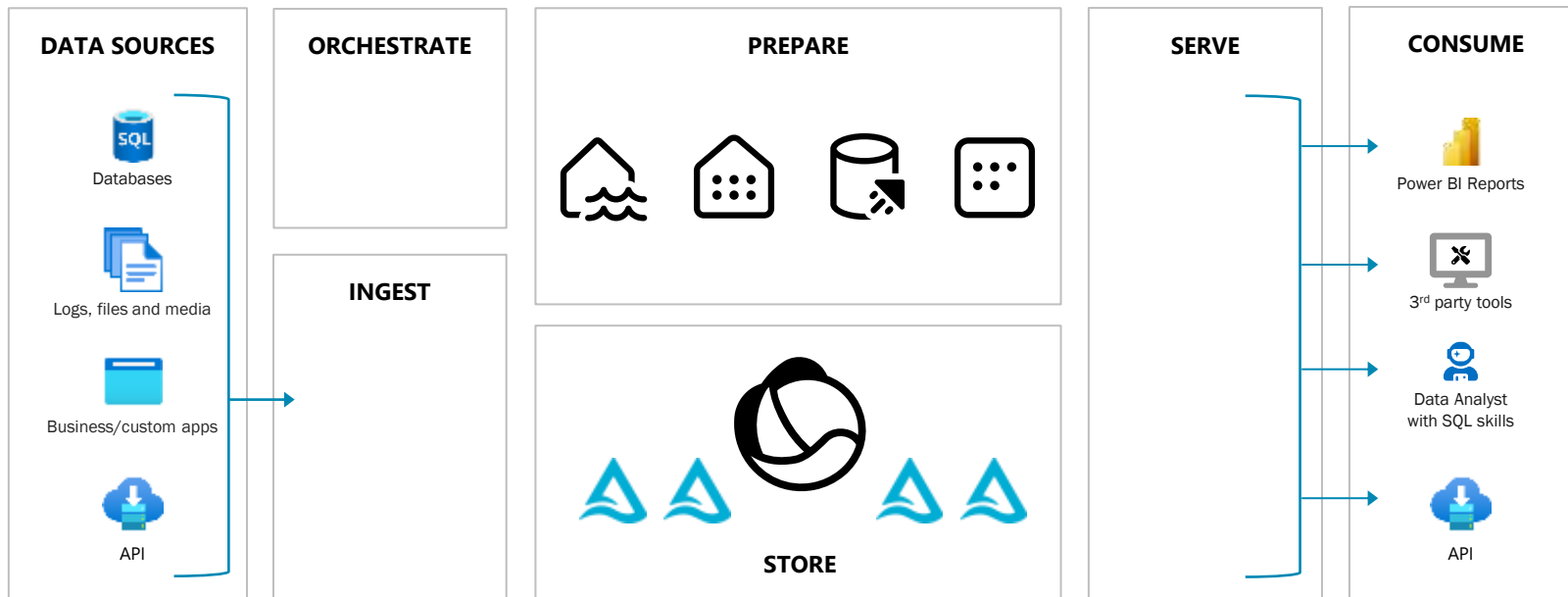


Separation of Compute and Storage





Separation of Compute and Storage





Separation of Compute and Storage

1. Storage is Cheap, Compute is Expensive

- Decouple to scale compute only when needed, reducing costs by avoiding over-provisioning.

2. Scalability Across Multiple Compute Engines

- Handle big data processing with Spark, analyze real-time events with Eventhouse, and scale each engine independently for maximum efficiency.

3. Tailor Compute Resources to Workloads

- Optimize performance by assigning the right compute engine—Spark, Data Warehouse, Eventhouse, or Analysis Services—to the appropriate tasks.

4. Always-On Storage with OneLake

- Keep your data accessible even when compute is inactive, ensuring persistent storage with minimal downtime.

5. Run Compute Where it Makes Sense

- Leverage compute engines in different environments while keeping your data centralized in OneLake, enabling seamless cross-cloud operations.



Delta Lake: The Magic Behind OneLake

- **The Backbone of Fabric's Lakehouse Architecture:** Delta Lake is the open-source storage layer that powers the Lakehouse, seamlessly integrating with engines like Spark for ultimate flexibility.
- **Default Storage Format for All Workloads in Fabric:** No matter what you're doing—streaming, batch processing, or analytics—Delta Lake is your go-to format.
- **ACID Transactions & Scalable Metadata Handling:** Go beyond Parquet! Delta Lake adds a transaction log for ACID compliance and robust metadata management, making it perfect for massive data environments.
- **Time Travel Made Easy:** Effortlessly access and revert to previous data versions for audits, rollbacks, or historical analysis. Delta Lake brings time travel to your data!
- **Turn Files into Relational Tables:** Files are no longer static! With Delta Lake, your files behave like relational tables, allowing for seamless querying and data manipulation.
- **"It's Parquet, but Better!"**





Delta Lake: The Magic Behind OneLake



DELTA LAKE

Name ↓

📁	_delta_log
📄	02403940

Name

📄	000000000000
📄	000000000000

```
{
  "add": {
    "path": "4374f6f8-f668-42ac-9ef9-fe85478de719.parquet",
    "partitionValues": {},
    "size": 173993,
    "modificationTime": 1696242397107,
    "dataChange": true,
    "tags": {}
  }
}
{
  "commitInfo": {}
}
```



DEMO

Delta Lake

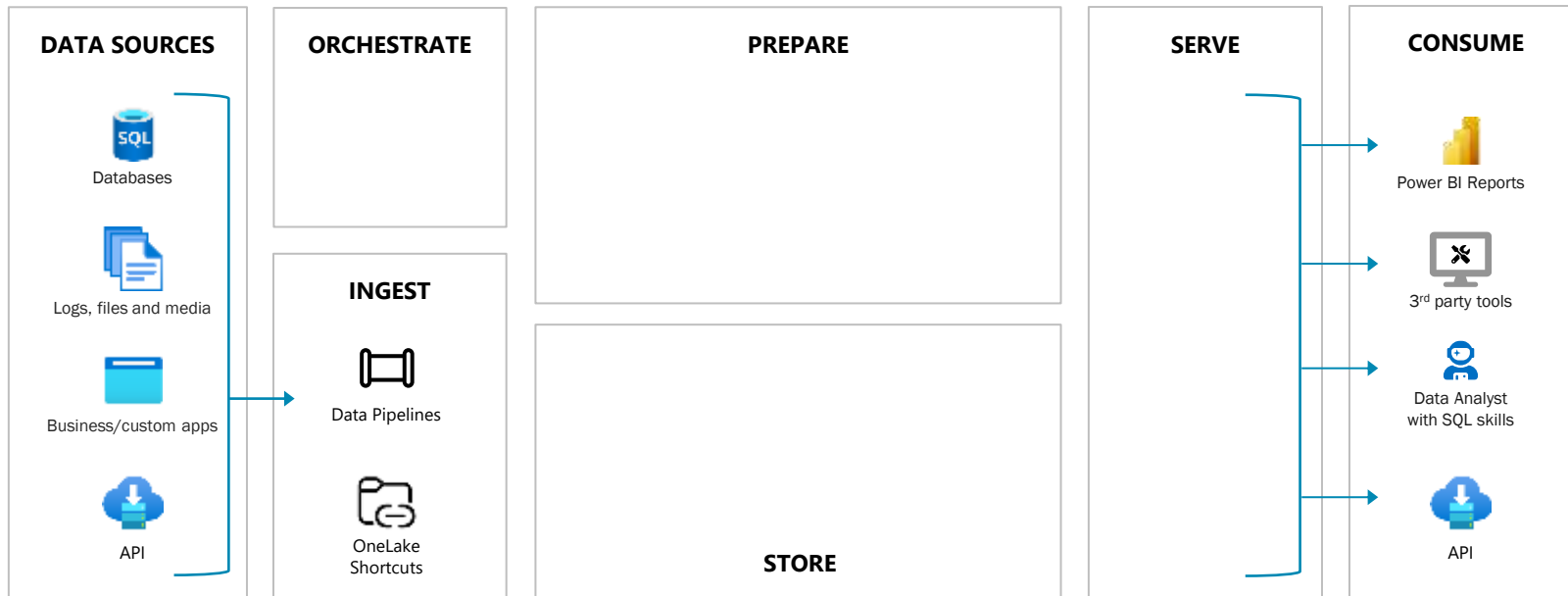


When PySpark Steals the Show

Key Scenarios for Success

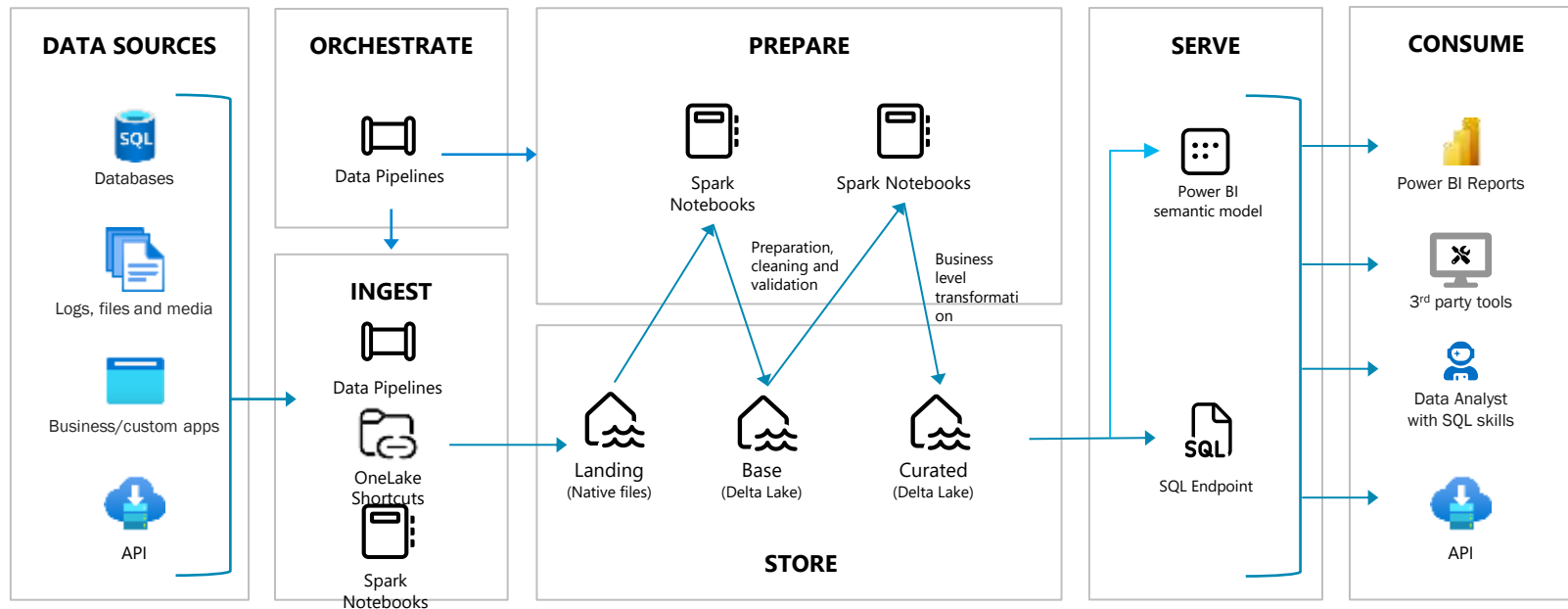


Dataflow in a Data Lakehouse Architecture



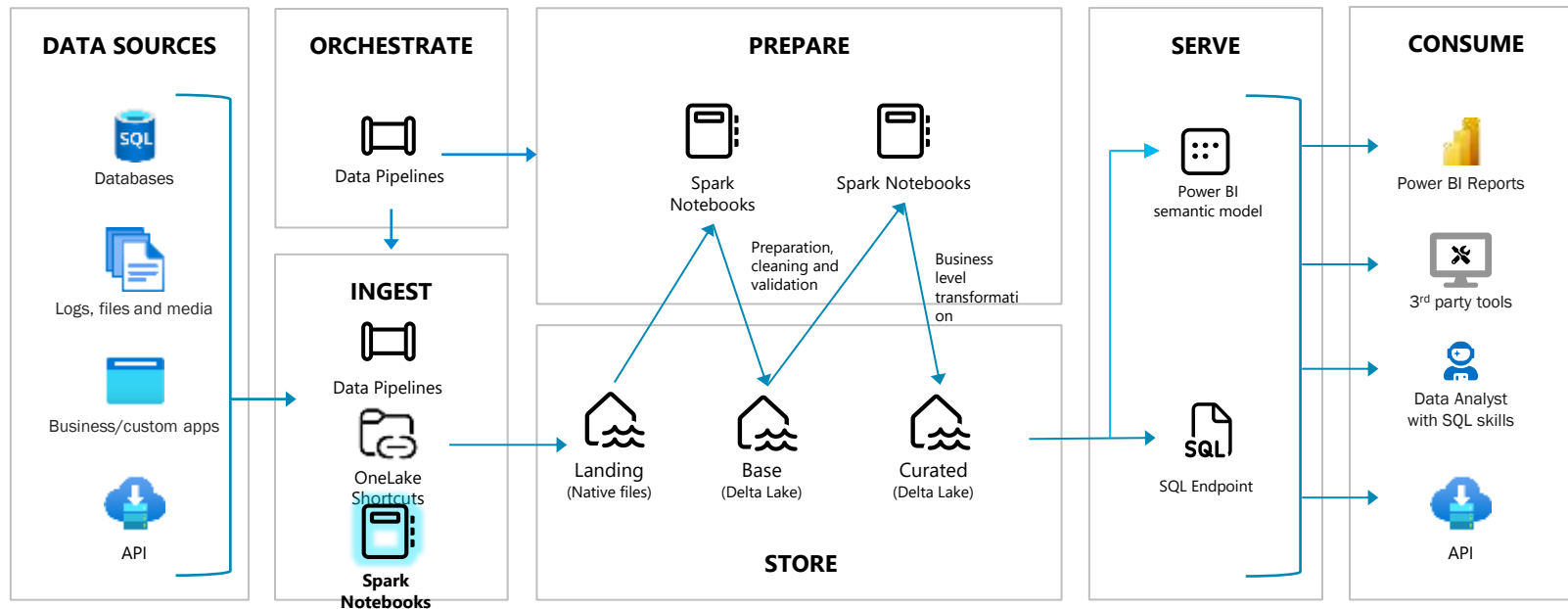


Dataflow in a Data Lakehouse Architecture



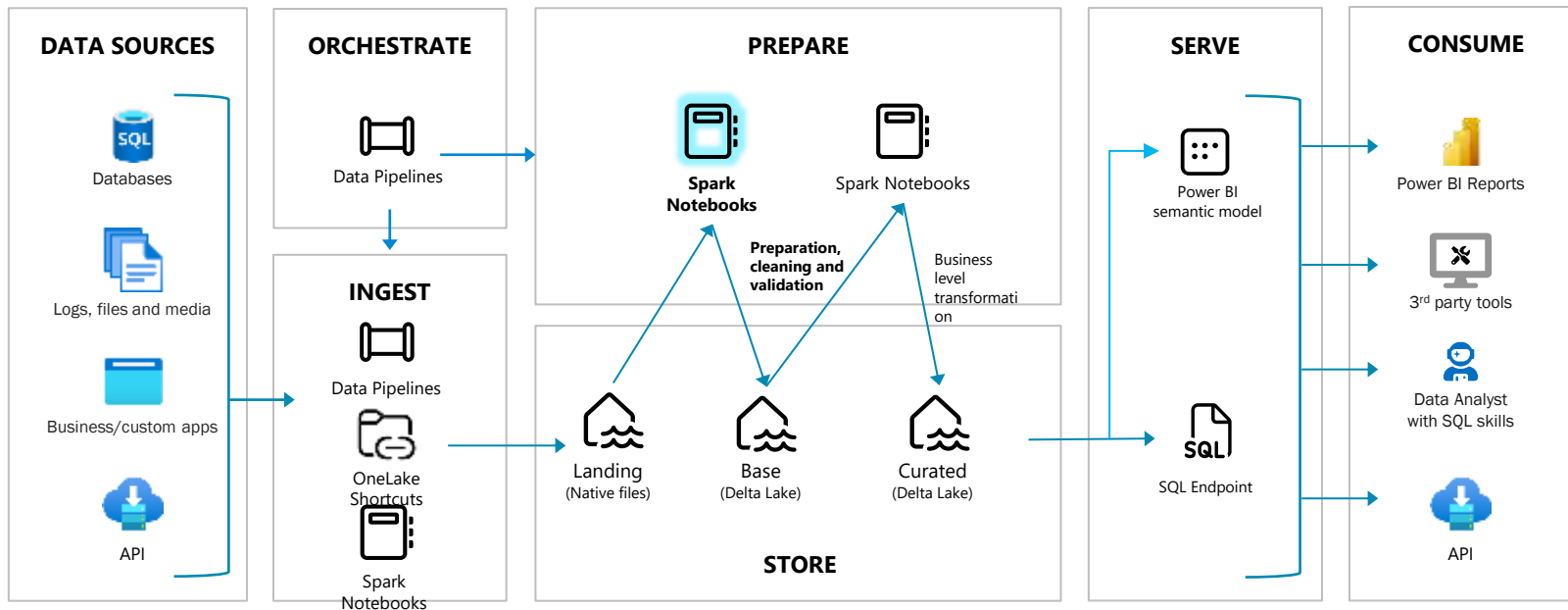


Scenario 1: Integration with Data Sources





Scenario 2: Automated Data Cleaning and Validation

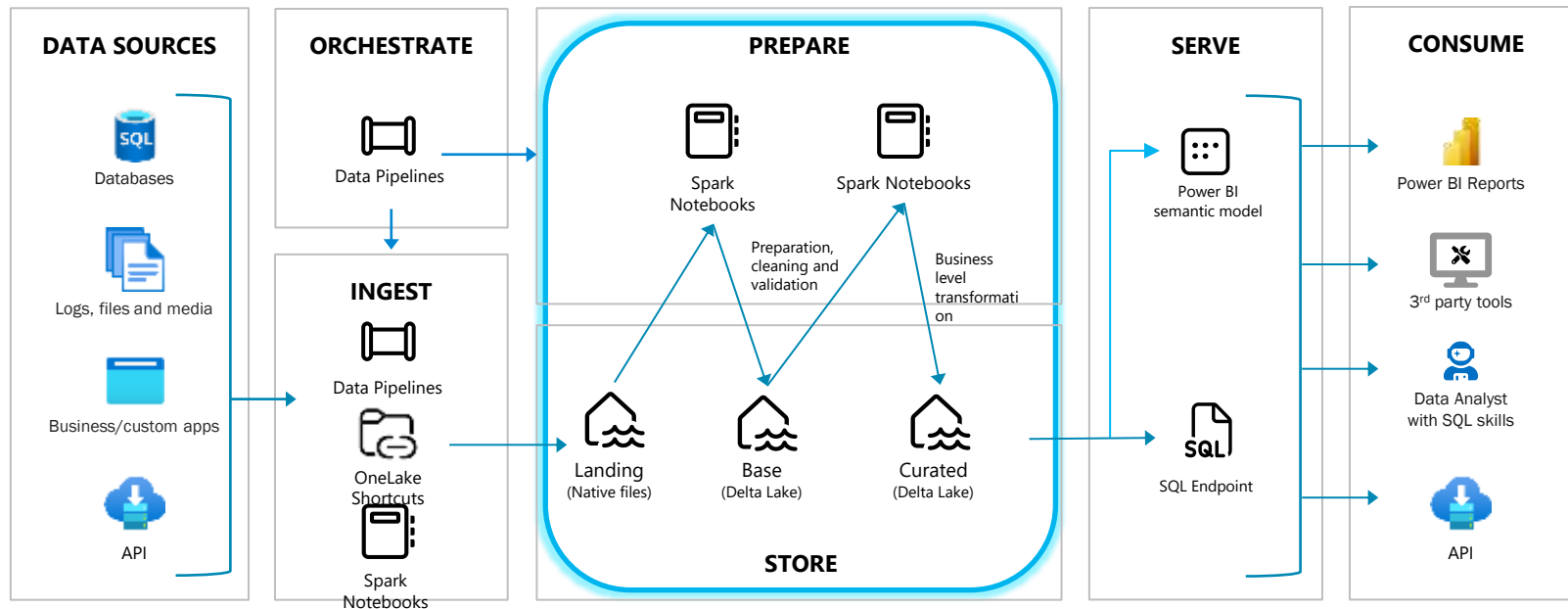


DEMO





Scenario 3: Breaking Free from SQL Constraints



DEMO



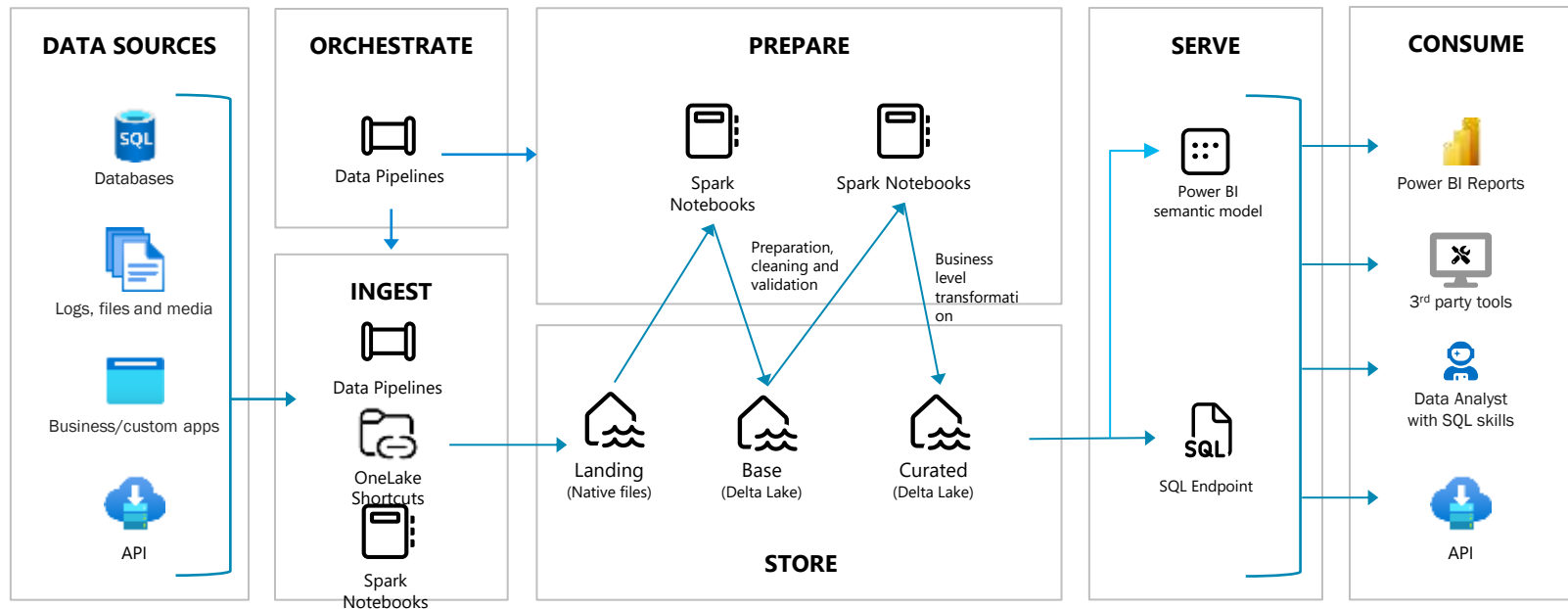


SQL is Still Cool

Why It's Here to Stay

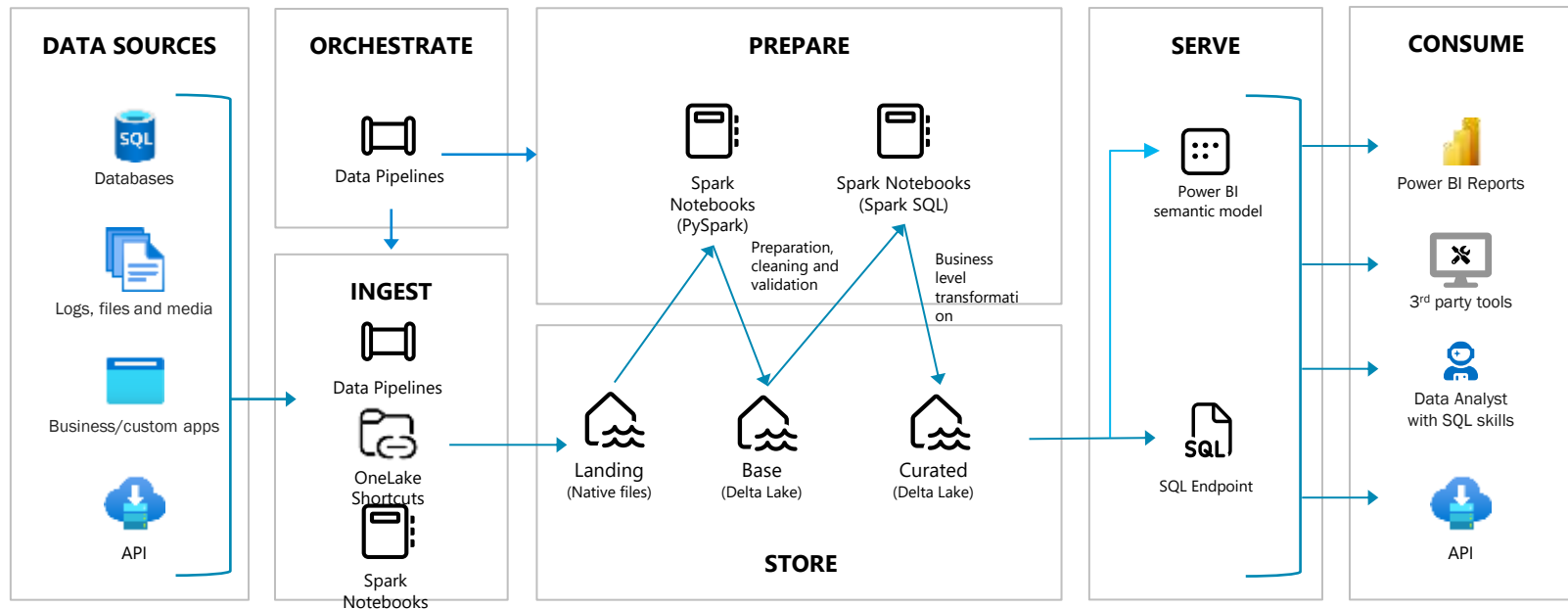


SQL's Cool Factor



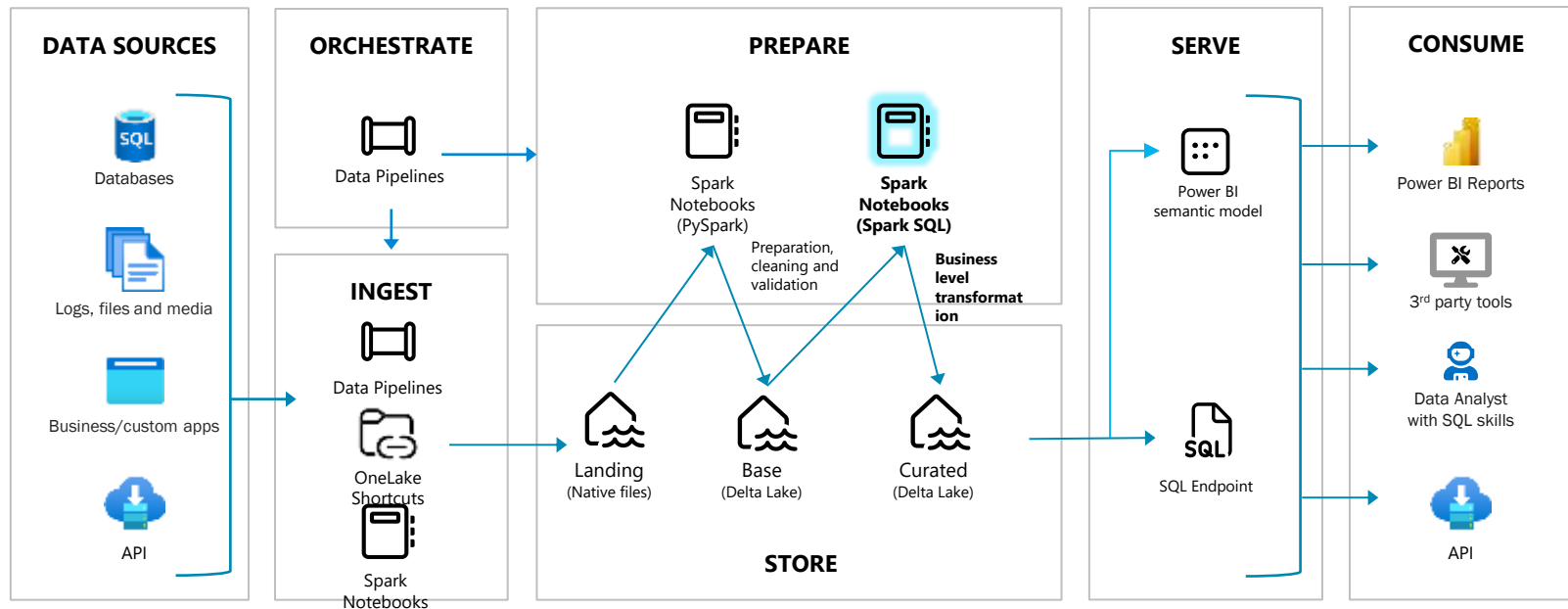


SQL's Cool Factor





SQL's Cool Factor



DEMO

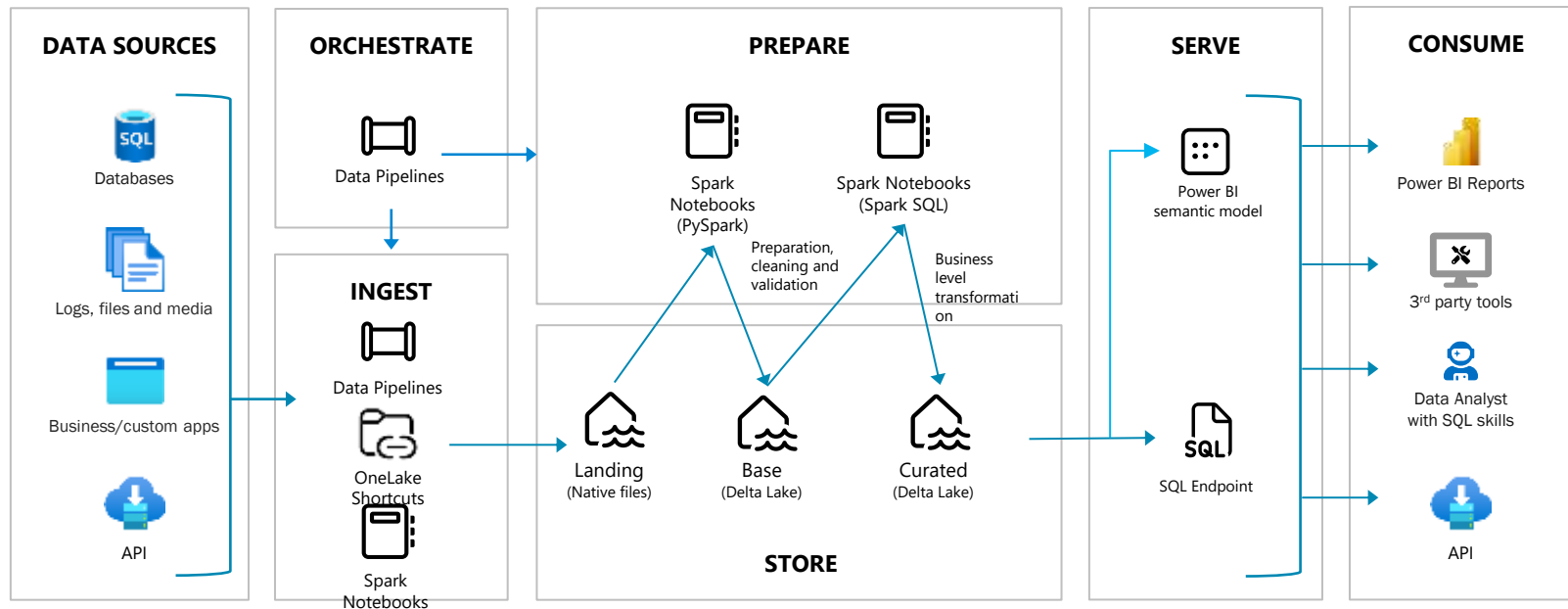


SQL's Cool Factor

- **Familiarity and Simplicity:** Leverage the power of a language you already know—SQL's declarative syntax is intuitive and widely adopted, making it easy for developers and analysts alike.
- **Seamless Transition to Modern Platforms:** SQL fits right into modern data ecosystems like Microsoft Fabric, allowing you to build on existing skills without the steep learning curve of new languages.
- **Powerful Performance with Spark SQL:** Execute SQL queries at scale with Spark SQL, merging the flexibility of SQL with the processing power of distributed systems like Spark.
- **Perfect for Business Logic:** SQL remains the gold standard for defining, refining, and executing business logic, ensuring clarity and precision in your data workflows.
- **Cross-Platform Portability:** SQL makes it simple to port business rules and queries across platforms, reducing the need for rework and ensuring consistency.
- **Integration with Modern Tools:** From Delta Tables to PySpark and Power BI, SQL plays well with today's most advanced tools, keeping it relevant and versatile.
- **SQL: The Glue for Data and Analytics:** SQL continues to bridge the gap between raw data and actionable insights, making it the connective tissue in any data architecture.

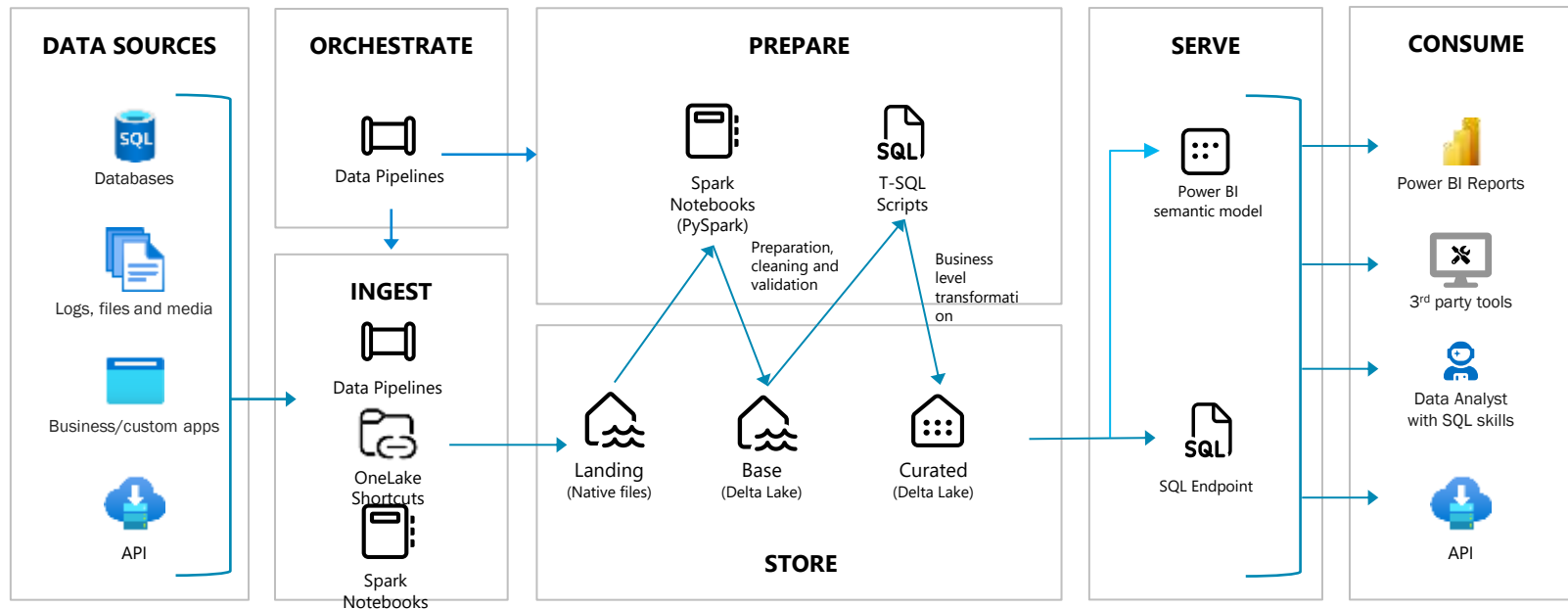


SQL is Still Cool



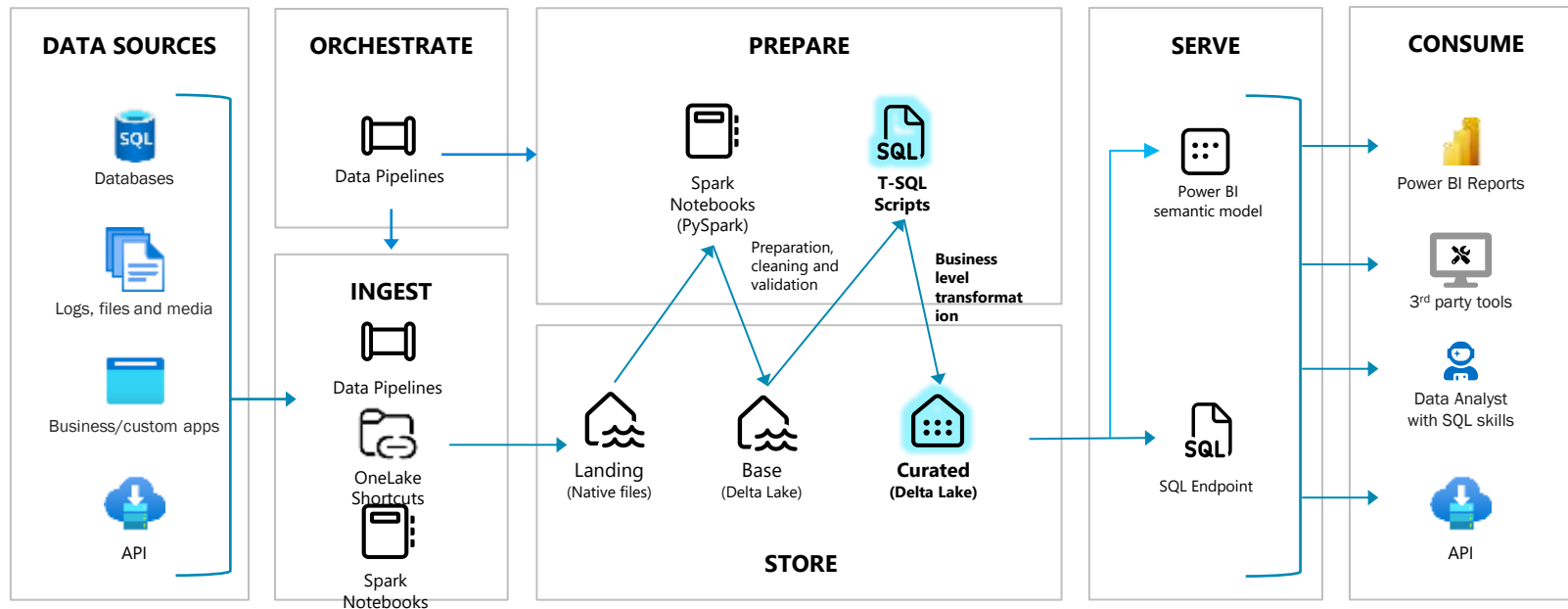


SQL is Still Cool





SQL is Still Cool





Session Feedback – THANK YOU 😊

Please evaluate this session using QR Code **during next 30 min**

- or -

Make your session feedback on your way out of the room

